

六、Agent Team：从单体到协同编排

核心问题： 单个 Agent 处理不了的大型任务，如何拆分给 Agent 团队协同完成？

OODA 定位： 从单体 OODA 到编队 OODA / Boyd 推论三回归

从上一节的问题切入

上一节结尾留了一个问题：

无论是流水线还是单 Agent 循环，都有一个共同限制——当任务复杂到一个上下文窗口装不下时怎么办？一个 Agent 搞不定的大任务，如何拆分给多个 Agent 协作？

上一节比较了 Agentless 流水线和 Agent 循环——Agentless 用 \$0.70 达到了 Agent 方案约 70-80% 的效果。但无论选哪条路线，都有一个共同的天花板：**单个上下文窗口**。当一个任务涉及前端 + 后端 + 测试的同时开发，单 Agent 的对话历史会被前一个子任务的中间状态污染——到第 25 轮时，早期的关键信息早已被稀释在数万 token 的工具结果中。

第五讲还埋了一个伏笔：CC 的 Subagent 机制本质上是 Pipeline 思维在 Agent 框架内的体现——主 Agent 规划拆解（类似流水线的 Localization）→ spawn 子 Agent 并行执行（类似 Agentless 的多候选并行采样）→ 结果汇总决策（类似 Validation 的筛选）。

本节从三个问题展开：**什么时候需要多 Agent？怎么拆任务？怎么防冲突？**

6.1 什么时候需要 Agent Team？——三个场景，不是万能药

Anthropic 官方博客（“When to use multi-agent systems”，2026.01）开篇就泼了一盆冷水：

“Most teams don’t need multi-agent systems.”

多 Agent 实现通常消耗单 Agent 的 **3-10 倍 token**，总执行时间甚至可能更长。“We’ve seen teams invest months building elaborate multi-agent

architectures only to discover that improved prompting on a single agent achieved equivalent results.” 只有三种场景下，多 Agent 一致优于单 Agent：

场景一：上下文污染（Context Pollution）

一个子任务产生的信息对后续子任务是**噪声**。

单 Agent 修 3 个模块（30+ 轮）：

Round 1-10: 处理 module_A → 积累 ~20K tokens (Read/Edit/test 结果)
Round 11-20: 处理 module_B → module_A 的中间推理仍在对话历史中
→ 注意力被无关信息稀释，推理质量开始下降
Round 21-30: 处理 module_C → 历史已 ~60K tokens
→ 早期信息被 auto-compact 压缩，关键细节可能丢失

3 个子 Agent 各修 1 个模块：

Worker A: 处理 module_A (干净上下文, ~10 轮, ~20K tokens)
Worker B: 处理 module_B (干净上下文, ~10 轮, ~20K tokens)
Worker C: 处理 module_C (干净上下文, ~10 轮, ~20K tokens)

→ 每个 Worker 的上下文都不会触发 auto-compact
→ 推理质量恒定，互不干扰

CC 源码实证（`src/services/compact/autoCompact.ts` 行 72）：auto-compact 阈值 = contextWindow - 33K tokens。即使有 6 层压缩纵深（第三讲 § 3.4），压缩必然有损——第三讲的 9 段摘要格式虽然优先保留 “Errors and fixes”，但跨模块的接口约定、参数命名等细节仍会丢失。**分治比压缩更根本：给每个子任务一个干净的独立窗口，从源头上避免污染。**

场景二：并行探索——彻底性，不是加速

Anthropic 官方原话：

“The primary benefit of parallelization is thoroughness, not speed.”

这一点很反直觉。多 Agent 并行不是为了缩短时间——事实上多 Agent 通常更慢（每个 Agent 需要独立的上下文构建、协调消息、结果汇总）。并行的真正价值是**覆盖更大的搜索空间**：

单 Agent 搜索（串行，深度优先）：

轮次有限 → 只能沿一条路径探索 → 可能错过最优解

N 个 Agent 并行搜索（广度优先）：

N 个独立方向同时调查 → 覆盖面更广 → 更可能找到正确答案

类比第五讲的 Agentless 多候选采样：

单次 $p \approx 0.2 \rightarrow 10$ 次并行 → 至少一个正确的概率 $\approx 89\%$

Claude Code 的 Research 功能就是这个模式：主 Agent 分析查询 → spawn 多个子 Agent 分别从不同角度搜索 → 各自返回精炼的发现 → 主 Agent 综合。

场景三：工具/领域专业化（Specialization）

当工具超过 20 个时，模型选错工具的概率显著上升。Anthropic 给出了三个信号：

1. **工具选择困难**：Agent 在 20+ 工具中挣扎，经常选错
2. **领域混淆**：数据库操作、API 调用、文件系统操作混在一起，Agent 分不清该用哪个领域的工具
3. **新工具干扰旧工具**：添加新工具后，原有任务的表现下降

CC 的解决方案是 **ToolSearch 延迟加载**（第二讲 § 2.3）——核心工具始终加载（~9 个），其余 25+ 个按需发现。Agent Team 则更进一步：每个子 Agent 只配备与其任务匹配的工具子集。

MVP 源码实证（`mvp/src/agent_tool.py`）：Worker 只能使用 Read/Write/Edit/Grep/Bash 共 5 个工具，**不能**调用 TaskCreate/TaskUpdate/Agent——从架构层面（而非提示词层面）阻止递归爆炸和角色越界。Worker 的职责是「执行」，不是「规划」。

小结：只在必要时引入多 Agent

决策框架：

1. 先用单 Agent + 好的 prompt 试
2. 如果遇到上下文污染 → 拆子 Agent 做隔离
3. 如果需要更彻底的搜索 → 并行子 Agent 覆盖更大空间
4. 如果工具太多选错 → 专业化子 Agent 聚焦工具集
5. 以上都不是 → 不要用多 Agent

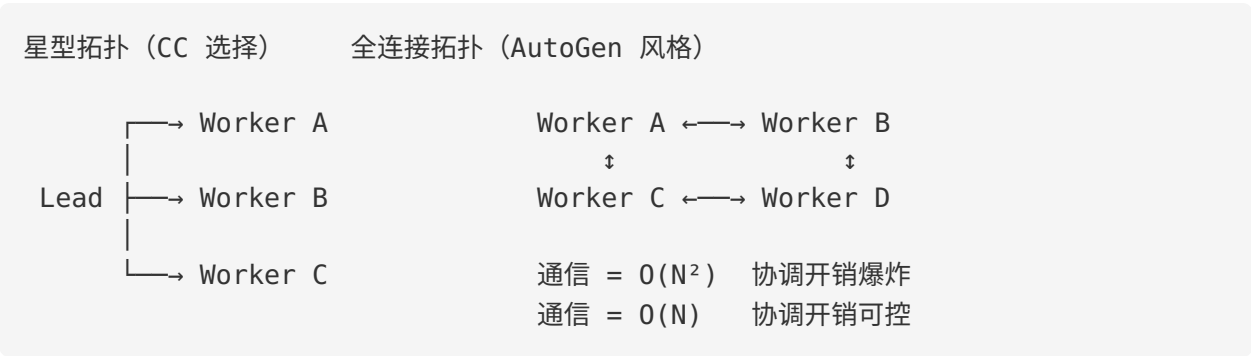
这与第一讲 Boyd 推论三一致——**协调开销决定编队 OODA 的上限**。每多一个 Agent 就多一份协调成本，所以只有在收益明确超过开销时才值得。

6.2 四种多 Agent 架构：从学术到工业

第一讲（§ 1.5）提到了三篇学术论文和 CC 的“第四条路”。现在展开对比：

[MetaGPT / AutoGen / OpenHands：三种多智能体范式]

维度	MetaGPT	AutoGen	OpenHands	Claude Code
编排方式	预设 SOP 流程	对话驱动自组织	Controller 硬编码调度	LLM 运行时自主编排
角色定义	固定（PM→架构师→SWE→QA）	预设 ConversableAgent	AgentDelegateAction	无预设，动态决定
通信拓扑	链式传递	灵活（支持群聊）	共享事件流	星型（仅与 Lead 通信）
通信开销	低（固定流程）	高（自由对话， $O(N^2)$ ）	中（Controller 中转）	$O(N)$ （星型拓扑）
隔离方式	部分隔离（共享消息池）	对话级隔离	Docker 沙箱隔离	完全隔离（独立上下文 + Git Worktree）
核心 trade-off	可预测 vs 僵化	灵活 vs 收敛困难	可控 vs 不灵活	灵活 vs 依赖 LLM 规划力



四个系统的核心分歧在一个设计决策上：**谁来决定 workflow？**

- MetaGPT：**人类**预定义 SOP（PM→架构师→SWE→QA），固定角色固定流程——如果任务不符合这个 SOP（比如需要先写测试再写代码的 TDD 流程），就无法适应

- AutoGen: **Agent 之间的对话**自然涌现工作流——灵活但容易陷入无效讨论，Anthropic 博客观察到 “agents spent more tokens coordinating than executing”
- OpenHands: **Controller 代码硬编码**调度逻辑——每种新的工作流都需要改代码
- CC: **LLM 自己在运行时决定**——通过工具调用组合出工作流，无需预定义

CC 的选择与它的整体设计哲学一致：**harness 提供能力（工具），model 提供智能（何时用、怎么用）**。

6.3 CC 的 Orchestrator-Worker 架构：协调原语工具化

CC 没有单独的 “orchestration engine” 或 “workflow engine”。所有编排逻辑都是模型的 Thought + 工具调用：

协调原语工具化：

工具	职责	类比
TeamCreate/Delete	创建/销毁协作上下文	docker-compose up/down
TaskCreate+blockedBy	任务分解 + 依赖图	Makefile DAG
Agent（并行 ×N）	派生 Worker	fork()
SendMessage	双向通信	IPC / 消息队列
TaskUpdate	状态同步	共享内存
文件系统	结果传递	Pipe / 共享存储

这就是 CC 的核心设计哲学——**精华不在某个单一工具，而在于把所有协调原语都做成工具，让 LLM 自己编排**。没有固定 DAG，没有预设角色，没有硬编码的 if/else 路由。

三条 Spawn 路径

CC 源码实证（src/tools/AgentTool/runAgent.ts、forkSubagent.ts、spawnInProcess.ts）：

路径	触发条件	隔离方式	适用场景
Normal	指定 subagent_type	全新 API 调用，独立上下文	独立子任务（最常用）
Fork	未指定 type + fork 开启	继承父上下文，共享 prompt cache	需要父 Agent 上下文的子任务

路径	触发条件	隔离方式	适用场景
Teammate	Swarm 模式	tmux/in-process 进程隔离	长期驻留的协作者

Normal 路径最干净——全新 API 调用，独立上下文，零信息泄露；Fork 路径最经济——继承父对话历史，共享 prompt cache 省钱，但携带了父任务的上下文噪声；Teammate 路径最重量级——tmux/iTerm2 进程隔离或 in-process AsyncLocalStorage 隔离，支持长期驻留的团队成员。

完整 Team 生命周期

阶段 1 · 团队创建

TeamCreate → TaskCreate(×N) → TaskUpdate 设置 blockedBy 依赖

阶段 2 · 并行派发

单条消息调用 Agent(×N)，每个 Worker 是独立 API 会话

阶段 3 · 进度汇报

Worker 通过 SendMessage({to: "team-lead"}) 报告给主 Agent

阶段 4 · 结果汇总

主 Agent 读取 Worker 写入的文件，审查质量，汇总交付

阶段 5 · 清理收尾

TeamDelete 销毁团队配置

通信机制

CC 源码实证 (src/tools/SendMessageTool/SendMessageTool.ts) :

SendMessage 基于**文件级 mailbox**——每条消息写入文件系统，接收方从文件读取。Worker 完成后向 Lead 发送 <task-notification> XML 消息；支持 to: "*" 广播给所有 teammate。这是一个极其简单的 IPC 实现，但足够用——子 Agent 之间不需要实时通信，异步消息传递即可。

CC 源码实证 (src/utils/tasks.ts) :

任务存储为 JSON 文件 (~/.claude/tasks/<taskListId>/<taskId>.json)，使用 proper-lockfile 文件锁防止并发冲突。同一 Team 的所有 Agent 共享同一个任务列表——这是协调的“共享状态”。

MVP 源码实证 (mvp/src/team_tools.py) :

MVP 用文件系统 JSONL 消息队列 (/tmp/team/{agent_id}/inbox.jsonl) 替代 CC 的进程间 IPC。选择 JSONL

而非 JSON 数组是因为 JSONL 支持 append-only 写入，在并发场景下更安全。学生可以直接 `cat /tmp/team/lead/inbox.jsonl` 看到消息流。Harness 兜底机制：如果 Worker 未调 `SendMessage` 就 `end_turn`，`SubAgentRunner` 自动将 Worker 最终回答补发给 Lead——不靠模型自觉，靠代码兜底。

6.4 隔离机制：对话隔离 + 文件隔离

多 Agent 协作需要在两个维度上做隔离：

维度一：对话隔离 (Context Isolation)

每个子 Agent 有独立的对话历史

→ 防止上下文污染 (Worker A 处理后端的中间状态不影响 Worker B 的前端推理)

维度二：文件隔离 (File Isolation)

每个子 Agent 在独立的 Git Worktree 中工作

→ 防止并发写冲突 (Worker A 和 Worker B 同时修改 `config.yaml` 不会互相覆盖)

对话隔离

CC 源码实证 (`src/utils/forkedAgent.ts`, `createSubagentContext()`)：每个子 Agent 创建独立的上下文——克隆 `readFileState`、创建子 `AbortController`、隔离工具状态集合。在 Normal 路径下，子 Agent 看不到父 Agent 或兄弟 Agent 的任何对话历史——它带着干净的窗口开始工作，只看到自己的任务 prompt 和项目环境信息 (`working dir`、`git status`、`CLAUDE.md`)。

Anthropic 博客用了一个客服场景解释为什么隔离重要：主 Agent 处理技术问题，子 Agent 查询订单历史。如果订单查询结果 (2000+ tokens) 留在主 Agent 的上下文中，会稀释主 Agent 对技术问题的推理注意力。子 Agent 查完后只返回 50-100 tokens 的提炼摘要——主 Agent 的上下文保持干净。

Git Worktree 文件隔离 (Worktree ≠ Branch)

一个常见误解：为什么不直接用 branch？因为 **branch 是逻辑隔离 (提交历史)**，**worktree 是物理隔离 (文件系统)**。

Branch 的问题：一个工作目录只能 checkout 一个 branch

Worker A: `git checkout branch-a` → 修改 `config.yaml`

Worker B: `git checkout branch-b` → 整个目录的文件被切换！

Worker A 正在编辑的 `config.yaml` 被替换成 branch-b 的版本

Worktree 的解法：每个 branch 一个独立目录

```
/project/                ← main (主 Agent 的工作目录)
/project/.claude/worktrees/a/ ← branch-a 的完整文件副本
/project/.claude/worktrees/b/ ← branch-b 的完整文件副本

→ Worker A 在 worktrees/a/ 里改 config.yaml
→ Worker B 在 worktrees/b/ 里改 config.yaml
→ 两个目录完全独立，互不干扰
→ 完成后：git merge branch-a branch-b → main
```

CC 的 `isolation: "worktree"` 参数自动调用 `git worktree add`，为每个子 Agent 创建独立的目录副本。**并发写的前提是物理隔离——branch 管版本历史，worktree 管文件系统。**

[OpenHands (2407.16741)：沙箱隔离实践]

OpenHands 走了更彻底的路线——为每个 Agent 提供独立的 Docker sandbox（独立文件系统 + 独立 shell 进程）。CC 的 Git Worktree 方案更轻量：不需要 Docker，利用 Git 原生分支机制实现文件隔离，merge 时自动检测冲突。轻量的代价是隔离不如 Docker 彻底——Worker 仍然共享主机的进程空间和网络。

权限冒泡

CC 源码实证（`src/utils/permissions/permissions.ts`）：子 Agent 的权限模式为 `bubble`——当子 Agent 遇到需要确认的危险操作（如删除文件），它不会直接询问用户，而是将权限请求**冒泡给父 Agent**。父 Agent 可以批准、拒绝或再冒泡给用户。这解决了“N 个并发子 Agent 同时弹确认框”的 UX 灾难。

6.5 模型路由：编排用强模型，执行用弱模型

Agent Team 中自然产生了一个模型选择问题：Lead 和 Worker 需要用同一个模型吗？直觉上，Lead 负责任务拆解和质量评估——这是高认知负荷的工作，需要强模型；Worker 负责在明确范围内写代码——这是执行性工作，弱模型即可胜任。

CC 的源码证实了这一直觉，并且把模型路由做成了一套**确定性分发规则**——不是“智能判断何时该用哪个模型”，而是基于订阅层级 + 用户配置 + 任务类型的三层路由。

默认模型选择：按订阅 tier 分级

CC 源码实证 (`cli.js`, `DN()` 函数)：默认模型的选择逻辑非常简单——Max 和 Team-5x 用户默认 Opus 4.6，其余所有用户默认 Sonnet 4.6。没有动态判断，没有任务分析，纯粹按订阅层级一刀切。

```
DN() {  
  if (isClaudeMax())      → Opus 4.6 (+ optional 1M context)  
  if (isTeamMax5x())      → Opus 4.6  
  return Sonnet 4.6       ← 所有其他用户的默认  
}
```

Sub-Agent 模型继承：默认跟随父模型

CC 源码实证 (`cli.js`, `JE6()` 函数)：Agent tool 的 `model` 参数默认值是 `"inherit"`——子 Agent 继承父 Agent 的主循环模型。用户可以通过 `model: "haiku"` / `model: "sonnet"` / `model: "opus"` 显式指定子 Agent 使用不同层级的模型。

```
JE6(requestedModel, mainLoopModel, override, permissionMode) {  
  if (CLAUDE_CODE_SUBAGENT_MODEL env) → 强制使用环境变量指定的模型  
  if (requestedModel == null)          → "inherit" → 继承父模型  
  if (requestedModel == "haiku")       → 解析为 Haiku 4.5  
  if (requestedModel == "opus")       → 解析为 Opus 4.6  
  ...  
}
```

这意味着 CC 当前的默认行为是 **Lead 和 Worker 用同一个模型**。但 `model` 参数的存在为“强 Lead + 弱 Worker”模式留了接口——用户可以手动指定 Worker 用 Haiku，Lead 继续用 Opus。

`opusplan` 模式：规划用 Opus，执行用 Sonnet

CC 源码实证 (`cli.js`, `hu()` 函数)：CC 有一个专门的 `opusplan` 别名，在 `plan` 模式下用 Opus 做深度推理，其余时段自动降级到 Sonnet：

```
hu({ permissionMode, mainLoopModel }) {  
  if (override == "opusplan" && mode == "plan")  
    return Opus 4.6    // plan 阶段：深度思考  
  if (override == "haiku" && mode == "plan")  
    return Sonnet 4.6  // haiku 用户在 plan 模式自动升级
```

```
return mainLoopModel // 执行阶段：保持原模型
}
```

`opusplan` 的设计哲学正是“规划用强模型，执行用弱模型”的工程化实现。花 \$5/M tokens 的 `Opus` 做任务拆解和架构决策（高认知负荷，token 少），花 \$3/M tokens 的 `Sonnet` 做代码编写和文件操作（低认知负荷，token 多），**整体成本接近纯 `Sonnet`，但规划质量接近纯 `Opus`。**

Haiku 的专属角色：轻量辅助任务

CC 源码实证（`cli.js`，`EJ()` 函数）：`Haiku 4.5` **不参与主对话循环**，专门用于轻量辅助任务：

使用场景	模型	成本
Hook prompt 回调处理	Haiku 4.5	\$1/M
WebFetch 内容摘要	Haiku 4.5	\$1/M
Web search 结果处理	Haiku 4.5	\$1/M
轻量 verification	Haiku 4.5	\$1/M
主循环（默认）	Sonnet 4.6	\$3/M
主循环（Max 用户）	Opus 4.6	\$5/M

这是一个三层模型金字塔：**Opus 做规划、Sonnet 做执行、Haiku 做杂活**——每一层匹配对应的认知负荷和成本预算。

对 Agent Team 设计的启示

CC 的模型路由揭示了一个工程原则：**模型选择不是全局统一的决策，而是按任务角色分级的分发策略**。这和我们在 § 6.1 讨论的“什么时候需要 Agent Team”是同一个问题的不同侧面——不仅上下文可以按角色隔离，**模型能力也应该按角色匹配**：

模型金字塔 × 角色匹配：

Opus 4.6 — Lead / Planner（任务拆解、质量评估、架构决策）
|
Sonnet 4.6 — Worker（代码编写、文件操作、测试执行）
|
Haiku 4.5 — Utility（内容摘要、格式校验、hook 回调）

回到 Demo 6 的可选演示——“编排需要强模型，执行可以用弱模型”——CC 的 opusplan 设计是这个论点的工业级实证：Anthropic 自己就认为规划和执行的认知负荷不同，值得用不同模型承担。

6.6 编队 OODA：嵌套循环

回到第一讲 Boyd 推论三：协调开销决定编队 OODA 的上限。

Agent Team 下，OODA 变成嵌套结构：

主 Agent 外循环（慢，粗粒度）：

- Observe 接收 Worker 的完成报告（ReadInbox / TaskList）
- Orient 评估整体进度，识别阻塞点和跨模块不一致
- Decide 增派 Worker？重新分配？直接 merge？
- Act spawn 新 Worker / 执行 merge / 运行集成测试

Worker 内循环（快，细粒度）：

标准的 Thought → Action → Observation 单 Agent 循环
在干净的独立上下文中运行，不受外循环干扰

L→R→V 也变成层次化执行：

Meta-Localization：Lead 分析需求 → 识别子系统 → TaskCreate 拆分子任务
↓
每个 Worker 独立执行自己范围内的 L → R → V
↓
Meta-Validation：Lead 汇总结果 → 运行集成测试 → 检查跨模块一致性

星型拓扑正是在这个嵌套结构中最小化协调开销：Worker 之间不通信（O(N)），只向 Lead 汇报。如果用全连接拓扑（AutoGen 风格），每个 Worker 都需要了解其他 Worker 的进度和接口——协调消息数量变成 O(N²)，Boyd 推论三预测的“协调开销吃掉收益”就会发生。

6.7 [Demo 6 • 预录+Live 混合] Agent Team 构建 Todo 服务

用一个真实的 Todo Web 应用构建任务，展示 Agent Team 的完整协作流程。这个任务已经在 MVP 上成功跑通（trajectory: session_20260330_201732.json），Lead 甚至发现了一个跨模块 bug。

任务：构建一个 Todo Web 应用（Flask 后端 + 原生 JS 前端）

主 Agent (Lead) 分析需求 → 拆解任务：



Lead 收到两个报告 → ReadInbox()

- 发现端口不一致（后端 5000，前端 fetch 指向 8000）
- spawn 新 Worker 修复前端端口
- 集成测试通过 → 交付

教学要点：Lead 发现端口 mismatch 正是 Agent Team 的 **Meta-Validation** 价值——两个 Worker 各自通过了单元测试（后端 API 响应正确、前端页面渲染正常），但跨模块的一致性只有 Lead 汇总时才能发现。这和现实中 Code Review 发现集成问题是同一个道理。

Demo 策略：

预录部分（3 分钟）：

播放 trajectory，展示完整 Team 协作流程——

Lead 拆解 → spawn backend-dev + frontend-dev →
各自在独立上下文中 L→R→V → SendMessage 汇报 →
Lead 发现端口 bug → spawn 修复 Worker → 交付

Live 部分 (2 分钟) :
用 MVP + Qwen3-30B-A3B 演示单个 backend-dev Worker
执行子任务。观察：
- Worker 的受限工具集 (只有 Read/Write/Edit/Grep/Bash)
- Worker 的独立上下文 (看不到 Lead 的对话历史)
- Worker 完成后 SendMessage 回报

可选演示：
让小模型充当 Lead 尝试分解任务，
观察其力不从心——任务分解不合理、Worker prompt 模糊。
引出：编排需要强模型，执行可以用弱模型。
"10 个实习生 + 1 个好的项目经理 = 高效团队；
10 个实习生 + 1 个实习生当经理 = 混乱。"

6.8 本节小结：从单体到编队的 OODA 演进

回到认知地图，本节将视角从单 Agent 扩展到多 Agent 编队——至此，课程的全部内容模块覆盖完毕：

	Localization	Repair	Validation
Observe (第四讲)	★ SWE-agent ACI ★ AutoCodeRover AST ★ Agentless 漏斗		
Orient (第三讲)	★ 测试报错 后的 Thought ★ 上下文压缩	★ Reflexion 经验积累	★ 死循环 诊断
Act (第二讲)		★ search/replace ★ CodeAct	
全局 (第五讲)	★ Agentless 三层漏斗 ← 流水线覆盖 L + R + V 全链路 →	★ 多候选 采样	★ 测试 筛选
★ 编队 (本节)	★ Meta-L Lead 拆解任务 ← 星型拓扑, O(N) 协调开销 →	★ 并发 Worker 独立执行	★ 集成 Lead 汇总验证

三个核心收获：

1. 三个场景，不是万能药

Anthropic 官方明确：多 Agent 系统消耗 3-10x token，总执行时间可能更长。只在三种场景下值得引入：上下文污染（子任务信息互相干扰，隔离是唯一根本解法）、并行探索（覆盖更大搜索空间，注意是**彻底性**而非加速）、工具/领域专业化（聚焦的工具集和 system prompt 提升可靠性）。“Start with the simplest approach that works, and add complexity only when evidence supports it.”

2. 协调原语工具化 = LLM 运行时自主编排

CC 的创新不在某个单一工具，而在于把 TeamCreate、TaskCreate、Agent、SendMessage **全部做成工具**，让 LLM 自己决定编排策略——没有固定 SOP（MetaGPT）、没有自由对话（AutoGen）、没有硬编码 Controller（OpenHands）。harness 提供能力（工具），model 提供智能（何时用、怎么用）。风险：这要求编排者是强模型——小模型做 Lead 力不从心，但做 Worker 绰绰有余。

3. Boyd 推论三的实证：少通信 > 多通信

“编队的整体 OODA 转速取决于协调开销。” CC 的星型拓扑将协调成本控制在 $O(N)$ ，每个 Worker 只向 Lead 汇报，不互相对话。对比 AutoGen 的全连接拓扑 ($O(N^2)$)，Anthropic 观察到 “agents spent more tokens coordinating than executing” ——**简单拓扑 > 复杂拓扑**。双层隔离（对话隔离 + Git Worktree 文件隔离）是并发协作的基础设施，缺一不可。

下一个环节：我们花了六讲拆解 Agent 的引擎盖——但 Agent 越强大，安全问题越突出。当 Agent 可以 spawn 子 Agent、并发修改文件、执行 shell 命令时，如何确保它不会搞出灾难？接下来 10 分钟，我们看看 CC 的权限引擎、Hook 系统和 CLAUDE.md 如何为 Agent 画出安全边界，然后做一个完整的双主线回顾。

加餐预告：六讲手工拆解的 system prompt、工具接口、上下文管理、循环控制——这些设计目前都是人类工程师手工调优的。但最新的研究表明：Harness 本身也可以被自动优化。Meta-Harness 论文用一个 coding agent 来搜索优化另一个 agent 的 harness code，在 TerminalBench-2 上击败了所有手工设计的

Haiku 4.5 方案。如果时间允许，我们看看这个 “Agent 优化 Agent” 的前沿方向。